

Tutorial Completo: Armazenamento Local no Navegador (localStorage e sessionStorage)

Para iniciantes em JavaScript

1. O que é localStorage?

`localStorage` é um objeto nativo do navegador que permite salvar dados no computador do usuário de forma persistente (os dados ficam salvos mesmo depois de fechar o navegador ou desligar o computador).

- Armazena apenas strings
- Capacidade geralmente entre 5-10 MB por domínio
- Os dados ficam disponíveis para sempre (até serem removidos manualmente)
- Funciona apenas no lado do cliente (nunca vai para o servidor)
- É compartilhado entre todas as abas/janelas do mesmo site

2. Métodos principais do localStorage

Método	O que faz	Exemplo	
<code>setItem(key, value)</code>	Salva um valor com uma chave	<code>localStorage.setItem("nome", "Maria")</code>	
<code>getItem(key)</code>	Recupera o valor de uma chave	<code>localStorage.getItem("nome")</code> → "Maria"	
<code>removeItem(key)</code>	Remove um item específico	<code>localStorage.removeItem("nome")</code>	
<code>clear()</code>	Remove TODOS os itens do localStorage	<code>localStorage.clear()</code>	
<code>key(index)</code>	Pega a chave pelo índice (0, 1, 2...)	<code>localStorage.key(0)</code>	
<code>length</code>	Quantidade de itens salvos	<code>localStorage.length</code>	

Exercícios de fixação (faça no console do navegador):

Exercício 1

Salve seu nome completo, idade e cidade usando localStorage. Depois recupere e mostre no console.

JavaScript



Copiar

```
localStorage.setItem("nomeCompleto", "Ana Silva");
localStorage.setItem("idade", "25");
localStorage.setItem("cidade", "São Paulo");

console.log(localStorage.getItem("nomeCompleto"));
console.log(localStorage.getItem("idade"));
console.log(localStorage.getItem("cidade"));
```

Exercício 2

Salve um contador de visitas. Toda vez que rodar o código, aumente em 1.

JavaScript



Copiar

```
let visitas = localStorage.getItem("visitas");
if (!visitas) {
  visitas = 0;
}
visitas = Number(visitas) + 1;
localStorage.setItem("visitas", visitas);
console.log("Você visitou esta página " + visitas + " vez(es)");
```

Exercício 3

Crie um botão que alterna um tema escuro (dark mode). Salve a preferência.

HTML

✕  Copiar

```
<button onclick="trocarTema()">Trocar Tema</button>

<script>
function trocarTema() {
  let temaAtual = localStorage.getItem("tema");
  if (temaAtual === "dark") {
    localStorage.setItem("tema", "light");
    document.body.style.background = "white";
    document.body.style.color = "black";
  } else {
    localStorage.setItem("tema", "dark");
    document.body.style.background = "black";
    document.body.style.color = "white";
  }
}

// Aplicar tema salvo ao carregar
let temaSalvo = localStorage.getItem("tema");
if (temaSalvo === "dark") {
  document.body.style.background = "black";
  document.body.style.color = "white";
}
</script>
```

3. O que é sessionStorage e como ele difere do localStorage?

Característica	localStorage	sessionStorage
Duração	Para sempre (até limpar)	Só enquanto a aba está aberta
Compartilhado entre abas	Sim	Não (cada aba tem o seu)
Uso comum	Preferências do usuário	Dados temporários da sessão

Exercícios de fixação:

Exercício 1

Abra duas abas do mesmo site. Salve algo no localStorage em uma aba e veja na outra. Depois faça o mesmo com sessionStorage.

JavaScript



Copiar

```
// Na aba 1
localStorage.setItem("teste", "visível nas duas abas");
sessionStorage.setItem("testeSessao", "só aparece nesta aba");

// Na aba 2 → localStorage vai mostrar, sessionStorage não
console.log(localStorage.getItem("teste"));          // funciona
console.log(sessionStorage.getItem("testeSessao")); // null
```

Exercício 2

Crie um contador que zera quando você fecha a aba (use sessionStorage).

JavaScript



Copiar

```
let cont = sessionStorage.getItem("contadorSessao");
if (!cont) cont = 0;
cont = Number(cont) + 1;
sessionStorage.setItem("contadorSessao", cont);
document.body.innerHTML += "<h1>Você recarregou esta aba " + cont + " vezes</
```

4. Trabalhando com dados complexos: JSON.stringify() e JSON.parse()

Como localStorage só aceita strings, usamos JSON para salvar objetos e arrays.

JavaScript



Copiar

```
// Objeto complexo
const usuario = {
  nome: "João",
  idade: 30,
  interesses: ["música", "programação", "games"],
  ativo: true
};

// Salvando
localStorage.setItem("usuario", JSON.stringify(usuario));

// Recuperando
const usuarioSalvo = JSON.parse(localStorage.getItem("usuario"));
console.log(usuarioSalvo.nome); // João
```

5. Exemplo prático: Lista de links favoritos persistente

HTML



Copiar

```
<!DOCTYPE html>
<html>
<head>
  <title>Meus Favoritos</title>
</head>
<body>
  <h1>Meus Links Favoritos</h1>
  <input type="text" id="url" placeholder="Cole a URL aqui">
  <input type="text" id="titulo" placeholder="Título do link">
  <button onclick="adicionar()">Adicionar</button>

  <ul id="lista"></ul>

  <script>
function carregarFavoritos() {
  const favoritos = JSON.parse(localStorage.getItem("favoritos") || "[]");
  const lista = document.getElementById("lista");
  lista.innerHTML = "";
  favoritos.forEach(fav => {
    const li = document.createElement("li");
    li.innerHTML = `<a href="${fav.url}" target="_blank">${fav.titulo}</a>
                  <button onclick="remover('${fav.url}')">Remover</button>`;
    lista.appendChild(li);
  });
}

function adicionar() {
  const url = document.getElementById("url").value;
  const titulo = document.getElementById("titulo").value || url;

  if (!url) return alert("Digite uma URL");

  let favoritos = JSON.parse(localStorage.getItem("favoritos") || "[]");
  favoritos.push({ url, titulo });
  localStorage.setItem("favoritos", JSON.stringify(favoritos));

  document.getElementById("url").value = "";
  document.getElementById("titulo").value = "";
}
```

```

    carregarFavoritos();
}

function remover(urlParaRemover) {
    let favoritos = JSON.parse(localStorage.getItem("favoritos") || "[]");
    favoritos = favoritos.filter(fav => fav.url !== urlParaRemover);
    localStorage.setItem("favoritos", JSON.stringify(favoritos));
    carregarFavoritos();
}

// Carregar ao abrir a página
carregarFavoritos();
</script>
</body>
</html>

```

6. Considerações de segurança e boas práticas

- Nunca salve senhas, tokens ou dados sensíveis no localStorage/sessionStorage
- Dados podem ser vistos e alterados pelo usuário no DevTools
- Use apenas para preferências, tema, carrinho temporário, etc.
- Sempre valide dados ao recuperar (nunca confie 100% no que está salvo)
- Use try/catch ao fazer JSON.parse() para evitar erros

JavaScript



Copiar

```

function getSafe(key, valorPadrao) {
    try {
        const item = localStorage.getItem(key);
        return item ? JSON.parse(item) : valorPadrao;
    } catch (e) {
        console.error("Erro ao ler localStorage", e);
        return valorPadrao;
    }
}

```

Mini Projeto Final: Sistema de Login Simples

Parte 1 – Usando localStorage (login persistente)

Crie uma página com:

- Campo de email e senha
- Botão "Login"
- Após login bem-sucedido, mostrar "Bem-vindo, [email]" e um botão "Sair"
- Ao recarregar a página, o usuário deve continuar logado

HTML



Copiar

```
<!DOCTYPE html>
<html>
<head><title>Login com localStorage</title></head>
<body>
  <div id="loginForm">
    <h2>Login</h2>
    Email: <input type="email" id="email"><br><br>
    Senha: <input type="password" id="senha"><br><br>
    <button onclick="fazerLogin()">Entrar</button>
  </div>

  <div id="logado" style="display:none">
    <h2>Bem-vindo, <span id="usuarioNome"></span>!</h2>
    <button onclick="logout()">Sair</button>
  </div>

  <script>
function fazerLogin() {
  const email = document.getElementById("email").value;
  const senha = document.getElementById("senha").value;

  if (email && senha) {
    // Salva o usuário logado
    localStorage.setItem("usuarioLogado", JSON.stringify({ email, loginTime
    mostrarLogado();
  } else {
    alert("Preencha email e senha");
  }
}

function logout() {
```

```
localStorage.removeItem("usuarioLogado");
document.getElementById("logado").style.display = "none";
document.getElementById("loginForm").style.display = "block";
}

function mostrarLogado() {
  const usuario = JSON.parse(localStorage.getItem("usuarioLogado"));
  if (usuario) {
    document.getElementById("usuarioNome").textContent = usuario.email;
    document.getElementById("loginForm").style.display = "none";
    document.getElementById("logado").style.display = "block";
  }
}

// Verifica se já está logado ao carregar
window.onload = function() {
  mostrarLogado();
}
</script>
</body>
</html>
```

Desafio Final (mude para sessionStorage)

Agora modifique o projeto para usar `sessionStorage` ao invés de `localStorage`.

Pergunta para reflexão:

O que acontece agora quando o usuário fecha a aba ou o navegador?

E se ele abrir o site em outra aba?

Resposta esperada:

Com `sessionStorage`, o login dura apenas enquanto a aba estiver aberta. Ao fechar a aba, o usuário será deslogado automaticamente — ideal para aplicações que exigem maior segurança ou quando não queremos login persistente.

Parabéns! Você agora domina `localStorage` e `sessionStorage`! 🎉

↳ Introdução ao IndexedDB

↳ Versão mais concisa do tutorial